

Trabajo Practico

(Final Libre)

Pensando la solución:

- 1-Clases principales (con sus respectivas funcionalidades).
- 2-Como implementarlas.
- 3-Como realizar la interfaz.
- 4-Implementar interfaz.

Como pense la solucion:

- Como requiero saber cuántos vecinos vivos tiene cada celda y no se actualizan en tiempo real, es decir, lo hacen todas a la vez, voy a requerir saber cuántos vecinos tiene cada una en el momento actual, por ende, guardare la cantidad de cada una en cada celda correspondiente, así podremos actualizar cada una sin depender del estado de las demás a la hora de modificar la celda.
- Entonces, primero hacemos un recorrido para saber las condiciones de cada celda y luego actualizamos (pasamos a la siguiente generación).
- Luego entra la lógica de las celdas la cual se evalúa con una condición dependiendo su contexto va a vivir o morir.
- Las clases extras como latente y enferma surgen de manera aleatoria en tablero ya que tenía como condición no afectar a las otras clases como vivir o morir, ya que de implementarlas como hijas de estas cambiarían los constructores y armados de las primeras 2.

Clases Principales:

Juego:

La clase Juego existe para coordinar la ejecución del juego y actuar como intermediaria entre la interfaz gráfica, el tablero y las reglas. Recibe el tablero que se va a utilizar, las reglas que determinan cómo evoluciona y la ruta del archivo donde se guardarán las generaciones.

El atributo tablero mantiene una referencia al tablero actual sobre el cual se ejecutan las generaciones, mientras que reglas almacena las condiciones que determinan el cambio de estado de las celdas.

El atributo velocidad representa el tiempo, en milisegundos, que transcurre entre cada generación automática. Se inicializa en 1000, equivalente a un segundo.

El atributo `corriendo` permite saber si el juego se encuentra ejecutándose o detenido. Los métodos `iniciar()` y `detener()` modifican este estado, mientras que `estaCorriendo()` permite que la interfaz gráfica consulte su valor.

El atributo `sinCambio` permite detectar si una generación produjo modificaciones en el tablero. Esto sirve para determinar si el juego llegó a un estado estable.

El atributo `rutaArchivo` almacena la ubicación del archivo utilizado para guardar el tablero. De esta forma, cada vez que se genera una nueva generación, el estado actualizado puede guardarse en el mismo archivo.

El constructor recibe el tablero, las reglas y la ruta del archivo y los asigna a los atributos correspondientes de la clase.

El método `paso()` se utiliza para ejecutar una única generación del juego. Primero solicita al tablero que genere su siguiente estado utilizando las reglas y comprueba si hubo cambios. Luego guarda el nuevo estado del tablero en el archivo correspondiente.

El método `iniciar()` establece el juego como activo, mientras que `detener()` lo establece como detenido. Estos métodos son utilizados principalmente por los botones Start y Stop de la interfaz.

El método `estaCorriendo()` permite consultar si el juego está actualmente en ejecución.

El método `sinCambio()` permite consultar si el tablero quedó igual después de generar una nueva generación.

El método `setVelocidad()` permite modificar la velocidad del juego, mientras que `getVelocidad()` devuelve la velocidad actual para que pueda ser utilizada por la interfaz gráfica y su Timer.

Finalmente, `getTablero()` permite que la interfaz gráfica obtenga el tablero actual para poder acceder a sus celdas y representarlas visualmente.

Resumen:

La clase `Juego` existe para centralizar y coordinar la ejecución del juego, conectando la interfaz gráfica con el tablero, las reglas, la velocidad de ejecución y el guardado del archivo.

Celda (Viva y Muerta):

La clase abstracta Celda existe para representar las características y comportamientos que tienen en común todos los tipos de celdas del tablero. Se utiliza como clase padre de estados como Viva, Muerta, Enferma y Latente, permitiendo aplicar herencia y polimorfismo.

El atributo simbolo almacena el carácter utilizado para representar visualmente el estado de la celda, mientras que vecinos almacena la cantidad de celdas vecinas que se encuentran vivas.

El constructor de Celda recibe el símbolo y la cantidad de vecinos inicial y los almacena en los atributos correspondientes.

El método abstracto sigEstado() establece que todas las clases hijas deben definir cómo pasa una celda de su estado actual al siguiente estado. Al ser abstracto, la clase Celda no define directamente qué sucede, sino que cada tipo de celda implementa su propia lógica.

El método setVecinos() permite modificar la cantidad de vecinos de una celda, mientras que getVecinos() permite obtener dicha cantidad.

El método getSimbolo() permite recuperar el símbolo que representa actualmente a la celda.

Finalmente, el método abstracto estaViva() obliga a cada clase hija a determinar si su estado debe considerarse vivo o no. Esto permite que el tablero pueda consultar una celda sin tener que conocer específicamente qué tipo de objeto es.

En una frase: la clase Celda existe para definir una estructura común para todos los estados del tablero y permitir que cada estado implemente su propio comportamiento mediante herencia y polimorfismo.

Clase Viva

La clase Viva representa una celda que actualmente se encuentra en estado vivo. Hereda de Celda, por lo que reutiliza sus atributos y métodos comunes.

Su constructor recibe el símbolo y la cantidad de vecinos y los envía al constructor de la clase padre mediante super().

El método sigEstado() determina qué ocurre con una celda viva en la siguiente generación. Si se cumplen las condiciones establecidas por las reglas, devuelve una nueva celda Viva; en caso contrario, devuelve una nueva celda Muerta.

El método estaViva() devuelve true, indicando que una celda de este tipo debe considerarse viva al momento de contar los vecinos.

En una frase: Viva existe para representar el estado vivo de una celda y definir cómo evoluciona según las reglas del juego.

Clase Muerta

La clase Muerta representa una celda que actualmente se encuentra en estado muerto. También hereda de Celda y utiliza los atributos y comportamientos comunes definidos por la clase padre.

Su constructor recibe el símbolo y la cantidad de vecinos y los pasa al constructor de Celda.

El método sigEstado() determina si una celda muerta debe permanecer muerta o volver al estado vivo. Si se cumplen las condiciones de las reglas, devuelve una nueva Viva; de lo contrario, devuelve una nueva Muerta.

El método estaViva() devuelve false, indicando que una celda muerta no debe contabilizarse como vecina viva.

Resumen

Muerta existe para representar el estado muerto de una celda y determinar si puede permanecer muerta o revivir en la siguiente generación.

Tablero:

La clase Tablero existe para representar y administrar la estructura completa del tablero del juego, incluyendo sus dimensiones, las celdas que lo componen, la generación aleatoria, la carga y guardado de archivos, el conteo de vecinos y la generación de los siguientes estados del juego.

Los imports de BufferedReader, BufferedWriter, FileReader y FileWriter permiten que la clase pueda leer y escribir archivos de texto, mientras que IOException permite controlar los posibles errores producidos durante esas operaciones. Random se utiliza para generar estados aleatorios y para aplicar las probabilidades durante la evolución del tablero.

El atributo random contiene un objeto de la clase Random, utilizado para generar valores aleatorios necesarios tanto para crear un tablero inicial como para aplicar las probabilidades de enfermedad y latencia.

El atributo numeroExa almacena la cantidad exacta de vecinos vivos necesaria para que una celda latente pueda volver a estar viva. Se establece como final porque su valor no debe modificarse durante la ejecución del tablero.

El atributo `celdas` representa la matriz bidimensional de objetos `Celda` que contiene físicamente todos los elementos del tablero. Al utilizar `Celda` como tipo, la matriz puede contener cualquiera de sus clases hijas, como `Viva`, `Muerta`, `Enferma` o `Latente`.

Los atributos `probabilidad` y `probabilidadLatente` almacenan las probabilidades utilizadas durante la evolución del tablero. La primera determina la posibilidad de que una celda viva pase a estar enferma y la segunda determina la posibilidad de que una celda que no está viva pase al estado latente.

Los atributos `filas` y `columnas` representan las dimensiones del tablero y determinan el tamaño de la matriz de celdas.

El atributo `signos` contiene los símbolos utilizados para representar los diferentes estados de las celdas y permite centralizar su utilización dentro del tablero.

El constructor `Tablero()` recibe las dimensiones, los símbolos, el número de vecinos necesario para las células latentes y las probabilidades. Con estos datos inicializa las características del tablero y crea la matriz de celdas con las dimensiones correspondientes.

El método `inicializarRandom()` se utiliza para crear el estado inicial del tablero de manera aleatoria. Recorre todas las posiciones de la matriz y genera un número aleatorio entre 0 y 3, utilizando cada valor para crear una celda `Viva`, `Muerta`, `Enferma` o `Latente`.

El método `guardarArchivo()` permite persistir el estado actual del tablero en un archivo de texto. Primero guarda las dimensiones del tablero y posteriormente escribe una fila por cada fila de la matriz, utilizando el símbolo correspondiente a cada celda. Esto permite recuperar posteriormente el mismo estado del tablero.

El método `cargarArchivo()` realiza el proceso contrario a `guardarArchivo()`: lee un archivo de texto y reconstruye el tablero a partir de la información almacenada. Primero comprueba que el archivo no esté vacío y que contenga correctamente las dimensiones. Luego verifica que las dimensiones del archivo coincidan con las del tablero actual.

Después de validar las dimensiones, `cargarArchivo()` recorre cada fila del archivo y analiza cada carácter. Dependiendo del símbolo encontrado, crea una instancia de `Viva`, `Muerta`, `Enferma` o `Latente`. También verifica que cada fila tenga la cantidad correcta de columnas y que no existan estados desconocidos ni filas adicionales.

El método `contarVecinos()` permite calcular cuántos vecinos vivos tiene una determinada celda. Para hacerlo analiza las ocho posiciones que pueden rodearla:

arriba, abajo, izquierda, derecha y las cuatro diagonales. Las condiciones que comprueban los límites de la matriz evitan acceder a posiciones que no existen.

El método siguiente `Generacion()` es uno de los métodos principales del tablero, ya que genera el estado siguiente del juego. Primero calcula y almacena la cantidad de vecinos vivos de cada celda. Después crea una nueva matriz para representar la siguiente generación sin modificar directamente la generación actual.

Para cada celda se consulta mediante `Reglas` si cumple las condiciones necesarias para cambiar de estado. Además, se aplican las probabilidades de enfermedad y de transformación a latente. Finalmente, cada celda obtiene su nuevo estado mediante `sigEstado()` cuando no se produce ninguno de los cambios probabilísticos.

Durante este proceso se compara el símbolo de cada celda nueva con el símbolo anterior. Si al menos una celda cambió, la variable `cambio` pasa a `true`. Esto permite detectar si el tablero continúa evolucionando o si llegó a un estado estable.

Una vez calculada toda la generación, la matriz nueva reemplaza a la matriz anterior. El método devuelve `true` si hubo algún cambio y `false` si el tablero permaneció exactamente igual.

El método `getCelda()` permite que otras clases, principalmente la interfaz gráfica, obtengan una celda específica del tablero sin acceder directamente a la matriz interna. Esto mantiene el atributo `celdas` encapsulado.

El método `getFilas()` devuelve la cantidad de filas del tablero y permite que otras clases conozcan sus dimensiones.

El método `getColumnas()` devuelve la cantidad de columnas del tablero.

El método `mostrar()` se encuentra actualmente comentado porque la representación principal del tablero se realiza mediante la interfaz gráfica. Se conserva como una herramienta de prueba que permitiría imprimir el tablero directamente en la consola.

Resumen:

La clase `Tablero` existe para administrar toda la estructura y evolución del juego: contiene las celdas, genera el tablero inicial, cuenta los vecinos, calcula las generaciones siguientes y permite guardar y recuperar el estado mediante archivos.

Condición (si la cantidad es menor, igual o mayor al dato + AND para doble condición):

La clase abstracta Condición existe para establecer una estructura común para todas las condiciones que pueden aplicarse sobre una celda. No se utiliza directamente para crear objetos, sino que sirve como clase padre para las diferentes condiciones.

El método abstracto cumple() recibe una celda y devuelve un valor booleano que indica si dicha celda cumple o no con la condición. Al ser abstracto, cada clase hija debe definir su propia forma de comprobar la condición.

La clase MenosDe representa una condición que verifica si una celda tiene menos vecinos vivos que una cantidad determinada. Su atributo minimo almacena ese límite y el constructor permite establecerlo. El método cumple() obtiene la cantidad de vecinos de la celda y devuelve true cuando dicha cantidad es menor que el valor establecido.

La clase MasDe representa una condición que verifica si una celda tiene más vecinos vivos que una cantidad determinada. Su atributo max almacena ese límite. El método cumple() devuelve true cuando la cantidad de vecinos de la celda supera dicho valor.

La clase IgualDe representa una condición que verifica si una celda tiene exactamente una determinada cantidad de vecinos vivos. Su atributo igual almacena el número que debe coincidir y el método cumple() devuelve true cuando la cantidad de vecinos de la celda es exactamente igual a ese valor.

La clase And permite combinar dos condiciones diferentes en una sola condición. Sus atributos c1 y c2 almacenan las dos condiciones que deben evaluarse.

El constructor de And recibe las dos condiciones y las almacena para poder evaluarlas posteriormente.

El método cumple() de And devuelve true únicamente cuando las dos condiciones se cumplen al mismo tiempo, utilizando el operador lógico &&. De esta manera se pueden construir reglas más complejas combinando condiciones simples.

Por ejemplo, si se tiene:

```
new And(new MasDe(1), new MenosDe(4))
```

se está creando una condición que se cumple cuando una celda tiene más de 1 vecino y menos de 4 vecinos.

Relación entre las clases

La estructura utiliza herencia y polimorfismo. Condición define qué significa que una condición pueda evaluarse mediante cumple(), mientras que MenosDe, MasDe, IgualDe y And proporcionan diferentes implementaciones de ese comportamiento.

Esto permite que Reglas pueda trabajar con objetos de tipo Condicion sin necesitar saber específicamente qué clase concreta está utilizando.

En el caso de And, sus dos condiciones también son objetos de tipo Condicion, por lo que puede combinar cualquiera de las condiciones existentes.

Resumen:

Estas clases existen para representar y combinar las condiciones que determinan si una celda cumple los requisitos necesarios para cambiar de estado, utilizando herencia, polimorfismo y operadores lógicos para construir reglas más complejas.

Reglas:

La clase Reglas existe para centralizar las condiciones que determinan cómo deben evolucionar las celdas del tablero, es decir, si una celda viva continúa viva o si una celda muerta puede revivir.

La clase hereda de Condición, por lo que debe implementar el método cumple(). Esto permite utilizar Reglas como una condición más dentro de la estructura de condiciones del programa.

El atributo vive almacena la condición que determina si una celda que actualmente está viva debe continuar viva. En el constructor se crea utilizando un And, combinando las condiciones MasDe y MenosDe. De esta manera, se establece un rango de cantidad de vecinos dentro del cual una celda puede mantenerse viva.

El atributo revive almacena la condición necesaria para que una celda que está muerta pueda volver a estar viva. En este caso se utiliza IgualDe, por lo que la celda debe tener exactamente la cantidad de vecinos indicada.

El constructor recibe tres valores: menor, mayor e igual. A partir de ellos construye las condiciones que utilizará el juego. Por ejemplo, con `new Reglas(2, 3, 1)`, una celda viva permanece viva cuando tiene entre 2 y 3 vecinos, mientras que una celda muerta revive cuando tiene exactamente 1 vecino.

El método cumple (Celda celda) es el encargado de determinar qué condición debe evaluarse según el estado actual de la celda. Si la celda está viva, se evalúa la condición vive; si está muerta, se evalúa la condición revive.

Esto permite que Tablero no tenga que conocer directamente las condiciones específicas para vivir o revivir, sino que simplemente consulte a Reglas mediante cumple ().

Resumen:

Muerta existe para representar el estado muerto de una celda y determinar si puede permanecer muerta o revivir en la siguiente generación.

Clases secundarias (forman parte de la solución pero no son el núcleo de la solución):

Celdas Extra (enferma, latente):

Clase Enferma

La clase Enferma representa una celda que se encuentra en estado enfermo. Hereda de la clase abstracta Celda, por lo que comparte sus atributos y debe implementar los métodos abstractos definidos por ella.

El atributo random permite generar valores aleatorios, aunque actualmente no es utilizado directamente dentro de esta clase.

El constructor recibe el estado de la celda y la cantidad de vecinos y los envía al constructor de la clase padre mediante `super()`.

El método `sigEstado()` determina el estado de la celda en la siguiente generación. En este caso, una celda enferma siempre pasa al estado muerto, independientemente de la cantidad de vecinos que tenga. Por este motivo, devuelve directamente una nueva instancia de Muerta.

El método `estaViva()` devuelve true, ya que para el conteo de vecinos una celda enferma se considera viva.

En una frase: Enferma existe para representar una celda enferma que se considera viva para el conteo de vecinos, pero que obligatoriamente pasa a estar muerta en la siguiente generación.

Clase Latente

La clase Latente representa una celda que se encuentra en un estado latente. Hereda de Celda y agrega el atributo `numExacto`, que indica la cantidad exacta de vecinos vivos necesaria para que la celda pueda revivir.

El constructor recibe el estado, la cantidad de vecinos y el número exacto requerido para revivir. Los dos primeros valores son enviados al constructor de Celda, mientras que `numExacto` se almacena en la propia clase.

El método `sigEstado()` determina si la celda latente puede volver al estado vivo. Para hacerlo, compara la cantidad actual de vecinos con `numExacto`. Si ambas cantidades

coinciden, devuelve una nueva celda Viva; si no coinciden, la celda permanece Latente.

El método `estaViva()` devuelve `false`, ya que una celda latente no se considera viva al momento de contar los vecinos.

A diferencia de una celda Muerta, la celda Latente tiene una condición específica para revivir basada en un número exacto de vecinos. Por esta razón, conserva el atributo `numExacto` cuando se crea nuevamente para la siguiente generación.

Resumen:

Latente existe para representar una celda que permanece en espera hasta que alcanza exactamente la cantidad de vecinos necesaria para volver al estado vivo.

Símbolos:

La clase Símbolos existe para centralizar los caracteres utilizados para representar visualmente los diferentes estados de las celdas. De esta manera, los símbolos no quedan escritos directamente en todas las clases del programa, sino que se almacenan en un único objeto y pueden ser reutilizados.

Los atributos `vivan`, `muerta`, `enferma` y `latente` almacenan respectivamente el carácter utilizado para representar cada uno de los cuatro estados posibles de una celda.

El constructor recibe los cuatro caracteres y los asigna a sus respectivos atributos, permitiendo definir los símbolos que utilizará el tablero al momento de crear el objeto.

Los métodos `getViva()`, `getMuerta()`, `getEnferma()` y `getLatente()` permiten consultar los símbolos almacenados desde otras clases. Por ejemplo, `signos.getViva()` permite obtener el carácter que representa a una celda viva.

Los métodos `setViva()`, `setMuerta()`, `setEnferma()` y `setLatente()` permiten modificar los símbolos después de haber creado el objeto. Esto hace que los caracteres utilizados para representar los estados puedan ser modificados sin necesidad de cambiar las clases que utilizan dichos símbolos.

La utilización de esta clase también evita repetir directamente caracteres como 'O', 'X', 'E' o 'L' por todo el código. En lugar de eso, las clases reciben un objeto Símbolos y consultan el carácter correspondiente.

Resumen

la clase Símbolos existe para centralizar y administrar los caracteres que representan los estados de las celdas, evitando tenerlos repetidos directamente en el resto del programa.

Ventana (interfaz):

Clase Ventana

La clase Ventana existe para crear y administrar la interfaz gráfica del juego, permitiendo que el usuario interactúe con el tablero mediante botones y un control de velocidad. La clase hereda de JFrame, que es la clase de Swing utilizada para representar una ventana principal.

Las importaciones javax.swing permiten utilizar componentes gráficos de Swing, como ventanas, botones, paneles, temporizadores y controles deslizantes. La importación java.awt proporciona las clases necesarias para trabajar con gráficos y realizar el dibujo del tablero.

El atributo juego almacena una referencia al objeto Juego. Esto permite que la interfaz pueda iniciar, detener y avanzar el juego, además de modificar su velocidad.

El atributo panel representa el área de la ventana donde se dibuja visualmente el tablero.

El atributo timer almacena un temporizador de Swing que permite ejecutar automáticamente una nueva generación cada cierto intervalo de tiempo.

Constructor Ventana

El constructor recibe una instancia de Juego y la almacena en el atributo juego. De esta manera, la interfaz gráfica trabaja sobre el mismo juego que contiene el tablero y las reglas.

setTitle() establece el título que aparecerá en la ventana, mientras que setSize() establece sus dimensiones iniciales. setDefaultCloseOperation() determina qué sucede cuando el usuario cierra la ventana; en este caso, se termina la ejecución del programa.

El JPanel se crea mediante una clase anónima, es decir, una clase que se define directamente en el lugar donde se instancia sin darle un nombre propio. Esta clase hereda de JPanel y sobrescribe el método paintComponent().

El método paintComponent() es utilizado por Swing para dibujar el contenido del panel. Primero se llama a super.paintComponent(g) para limpiar correctamente el área

de dibujo y preparar el panel. Después se llama a `dibujarTablero(g)`, que contiene la lógica específica para representar el tablero.

El método `add(panel)` agrega el panel a la ventana para que pueda mostrarse.

Botones de control

Los objetos `JButton` representan los botones que permiten controlar el juego desde la interfaz.

El botón `Start` utiliza `addActionListener()` para detectar cuando el usuario lo presiona y llamar al método `iniciar()` de `Juego`.

El botón `Stop` realiza el proceso contrario y llama al método `detener()`.

El botón `Step` ejecuta manualmente una única generación mediante `juego.paso()`. Luego utiliza `panel.repaint()` para solicitar que `Swing` vuelva a dibujar el tablero y mostrar visualmente el nuevo estado.

Los `ActionListener` permiten asociar una acción a un evento producido por el usuario, como presionar un botón. La expresión `e ->` es una expresión lambda que representa la acción que se ejecutará cuando ocurra dicho evento.

Panel de controles

Se crea otro `JPanel` llamado `controles` para agrupar los botones y el control de velocidad.

Los botones `Start`, `Stop` y `Step` se agregan a este panel mediante `controles.add()`.

Control de velocidad

El `JSlider` permite al usuario modificar visualmente la velocidad de ejecución del juego.

El rango establecido va desde 100 hasta 2000, y su valor inicial se obtiene mediante `juego.getVelocidad()`.

Estos valores representan milisegundos de espera entre generaciones, por lo que un valor menor hace que el juego avance más rápido y un valor mayor hace que avance más lentamente.

`setMajorTickSpacing(500)` establece la separación entre las marcas principales del deslizador, mientras que `setMinorTickSpacing(100)` establece la separación entre las marcas secundarias.

`setPaintTicks(true)` hace visibles las marcas del deslizador y `setPaintLabels(true)` muestra los valores numéricos.

`JLabel("Velocidad:")` agrega un texto para indicar al usuario qué representa el deslizador.

Finalmente, el panel de controles se agrega a la parte inferior de la ventana mediante `BorderLayout.SOUTH`.

Temporizador Timer

El `Timer` de `Swing` se utiliza para generar automáticamente nuevas generaciones del juego después de cada intervalo de tiempo establecido.

El intervalo inicial se obtiene mediante `juego.getVelocidad()`.

Cada vez que el temporizador genera un evento, se verifica mediante `juego.estaCorriendo()` si el juego está actualmente iniciado. Si está corriendo, se ejecuta `juego.paso()` para calcular la siguiente generación y posteriormente `panel.repaint()` para actualizar la representación visual.

`timer.start()` inicia el temporizador, aunque las generaciones solamente se ejecutarán cuando `estaCorriendo()` devuelva `true`.

Cambio de velocidad

`velocidad.addChangeListener()` permite detectar cuando el usuario modifica el valor del `JSlider`.

Cuando esto ocurre, `getValue()` obtiene el nuevo valor seleccionado. Luego `juego.setVelocidad()` actualiza la velocidad almacenada en el objeto `Juego` y `timer.setDelay()` modifica inmediatamente el intervalo del temporizador.

Esto permite que la velocidad pueda modificarse mientras el juego está funcionando, sin necesidad de reiniciar el programa.

Método `dibujarTablero()`

El método `dibujarTablero()` tiene como única responsabilidad representar gráficamente el estado actual del tablero.

Primero obtiene el tablero desde el objeto `Juego` mediante `getTablero()`.

Después obtiene el ancho y alto disponibles del panel y consulta la cantidad de filas y columnas del tablero.

A partir de estas dimensiones calcula el ancho y alto que tendrá cada celda, dividiendo el tamaño disponible del panel por la cantidad de columnas y filas.

Luego utiliza dos ciclos `for` para recorrer todas las posiciones de la matriz del tablero.

Para cada posición obtiene la celda correspondiente mediante `getCelda(i,j)` y consulta su símbolo mediante `getSimbolo()`.

`g.drawString()` dibuja el símbolo de la celda dentro de la posición correspondiente del tablero.

Finalmente, `g.drawRect()` dibuja el borde de cada celda, permitiendo visualizar la matriz como una cuadrícula.

Resumen:

la clase `Ventana` existe para proporcionar la interfaz gráfica del juego, representar visualmente el tablero y permitir al usuario controlar su ejecución, avanzar generaciones y modificar la velocidad mediante componentes de Swing.

Main:

`Main` se encarga de iniciar el programa, pedir y validar los parámetros iniciales, crear el tablero y las reglas, decidir si el tablero se genera aleatoriamente o se carga desde un archivo, establecer la ruta de guardado y finalmente crear la interfaz gráfica.